Yes, that's essentially correct. Here's a clearer explanation:

STORE AC

- When the instruction (such as STORE AC) is fetched, it is stored in the Instruction Register (IR). The IR holds the complete instruction, including its address field (often shown as IR(address)).
- This address field specifies the memory location that the instruction will operate on. For STORE AC, it tells the processor where in memory to store the contents of the Accumulator (AC).
- During the execute cycle for STORE AC, the control unit takes the address from the IR and transfers it into the Memory Address Register (MAR). This is why you see the micro-operation:
  t1: MAR ← IR(address) (with control signal C8)
- The MAR now holds the target address for the memory operation, and the processor proceeds to move data (from AC to the Memory Buffer Register and then to memory) using subsequent micro-operations.

So, while the PC was used during the fetch cycle to retrieve the instruction, the IR now contains that instruction—with its address field instructing where the operation (like storing AC) should occur.

LOAD AC:

What the LOAD AC Instruction Does

- **Objective:**
  The LOAD AC instruction is designed to fetch (or load) data

from a specific memory location into the Accumulator (AC).

- **Where Is the Source Address?**
  During the fetch cycle the processor used the PC (Program Counter) to retrieve the instruction from memory. That fetched instruction is stored in the Instruction Register (IR). The IR contains not only the opcode (which tells the processor this is a LOAD AC instruction) but also an address field (often written as IR(address)). This address field tells the processor from which memory location the data should be loaded.

The Role of Each Register for LOAD AC

- **IR (Instruction Register):**
  Holds the fetched LOAD AC instruction, including the memory address where data must be read from.

- **MAR (Memory Address Register):**
  The processor uses MAR to specify the address from memory for the operand. For a LOAD AC instruction, MAR gets the address from the instruction in the IR.

- **MBR (Memory Buffer Register):**
  This acts as an intermediate data buffer; it temporarily holds the data read from memory before it is moved into the AC.

- **AC (Accumulator):**
  This register will ultimately receive the data read from the

specified memory location.

Step-by-Step Explanation of the Micro-Operations for LOAD AC

1. **t1: MAR ← IR(address) (Control signal: C8)**
   - **What It Does:**
     Here, the control unit takes the address part from the already-fetched instruction (found in IR(address)) and loads it into the MAR.
   - **Why:**
     The memory unit needs a destination address to know which memory location to read data from. Using IR(address) makes sure that we use the correct address specified by the instruction, not the PC (which was only used to fetch the instruction).

2. **t2: MBR ← Memory (Control signals: C5, CR)**
   - **What It Does:**
     Once the MAR is loaded with the appropriate address, the memory unit is accessed. The data stored in that memory location is fetched and loaded into the MBR.
   - **Why:**
     The MBR serves as a temporary holding area for the data coming from memory. This is necessary because the next step requires the data to be moved from the MBR into the AC.

3. **t3: AC ← MBR (Control signal: C10)**
   - **What It Does:**
     Finally, the data in the MBR is transferred into the

Accumulator (AC).
  - **Why:**
    The LOAD AC instruction's purpose is to place the fetched data into the AC so that later instructions can use it for computation.

Why Not Use the PC During Execution?

- **Role of the PC:**
  The Program Counter (PC) was used during the fetch cycle to point to the memory location containing the instruction. Once the instruction is fetched and stored in the IR, the PC is incremented to point to the next instruction.

- **Using IR Address for Operand Access:**
  For the execution of LOAD AC, the processor must access the operand from memory. The address for that operand comes from the IR (IR(address)) since the instruction specifies where to load from. The PC is not reused here because its job in the fetch cycle is complete.

In Summary

- **Fetch Cycle Completion:**
  The instruction (LOAD AC) was fetched using the PC, and is now safely in the IR. This cycle is over once the instruction is in the IR.

- **Execution Cycle (for LOAD AC):**
  - **Step t1:** Use the address in the IR to load the MAR.
  - **Step t2:** Fetch the data from memory, placing it into the MBR.
  - **Step t3:** Transfer the data from the MBR into the AC.

This sequence clearly shows that the address needed to load the data comes from the IR rather than the PC because the instruction already resides in the IR after the fetch cycle is done. The LOAD AC operation is part of the execution phase, not the fetch phase.